EE 271- ADVANCED DIGITAL SYSTEM DESIGN AND SYNTHESIS

PROJECT REPORT

16-BIT FIXED POINT MAC FOR AI ENGINE

SUBMITTED BY (Group 9) :
VASU ERANKI
PRIYANKA GOPINATH
SAI KASHYAP KURELLA

Google Drive Link for video: EE271_Project.mp4

Table of Contents

1. INTRODUCTION

1.1 FLOATING POINT NUMBER

A number that usually has a decimal point is called Floating-Point Number. For example, 4.15, -3.56 are called Floating-Point Numbers. Floating Point representation makes the numerical computation easier.

Floating Point representation is divided into 3 parts called Sign, Exponent, and Mantissa and all this part consists of several bits depending upon the precision type. According to IEEE 754 which is a standard for floating-point has issued 3 types of precisions and are Half Precision, Single Precision, and Double Precision. Half Precision consists of 16 bits, Single precision consists of 32 bits and Double precision consists of 64 bits.

1.2 HALF PRECISION FLOATING POINT REPRESENTATION

Half precision floating point representation (sometimes called FP16) is of 16 bits. The largest number that can be represented is $2^{15} - 1 = 32767$ and the smallest number that can be represented is -32767. The IEEE 754 standard specifies a binary16 as having the following format:

- Sign bit: 1 bit
- Exponent width: 5 bits
- Significand precision: 11 bits

2. HARDWARE

The list of the hardware components are:

1. DE-10 Lite FPGA Board.
2. 4x4 Keypad.
3. Arduino Uno Board.
4. LCD.

2.1 DE-10 LITE FPGA BOARD

DE10-Lite is a cost-effective Altera MAX 10-based FPGA board. The board utilizes the maximum capacity MAX 10 FPGA, which has around 50K logic elements (LEs) and on-die analog-to-digital converter (ADC). It features on-board USB-Blaster, SDRAM, accelerometer, VGA output, 2x20 GPIO expansion connector, and an Arduino UNO R3 expansion connector in a compact size. With all these features, this board was one of the best and economic choices to implement floating point adder. The board had six 7 segment displays in which both the inputs and the output of floating-point addition was displayed.

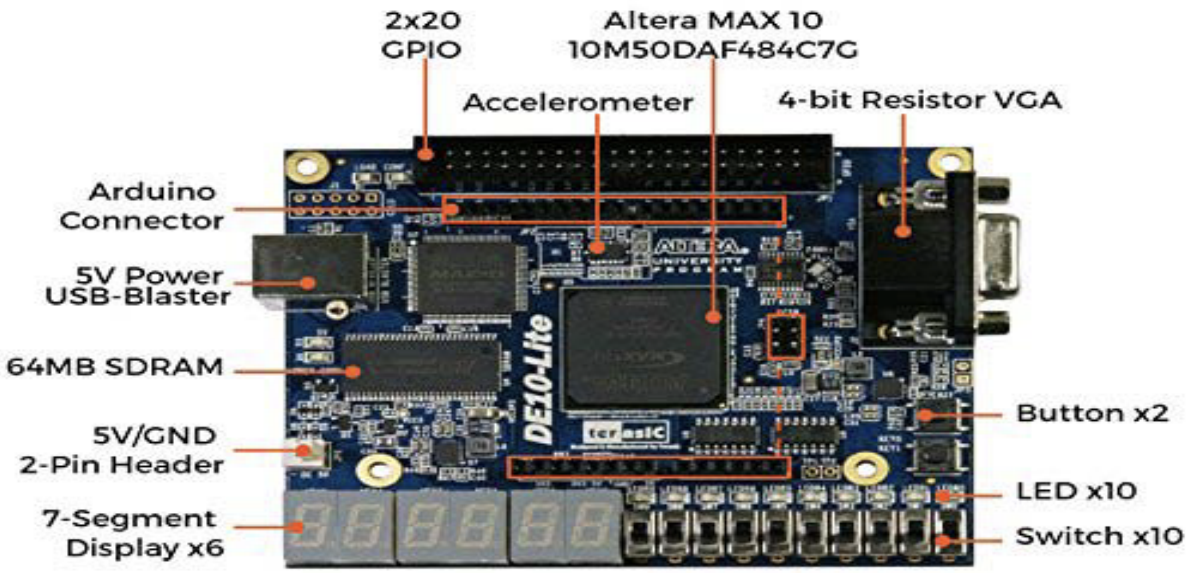

Figure 1. DE-10 LITE FPGA Board

2.2 4X4 KEYPAD

Pressing a button on the keypad shortens one of the column lines to one of the row lines for the current to flow between them. This is the basic principle behind the keypad. The keypad connected to the FPGA scans the lines to check if any column and row is low. For this initially, all the row and column lines are set HIGH. When a button is pressed, the associated row and column line goes LOW. FPGA scans each row and column line to check if any row or column line has gone LOW. This way, FPGA knows which button is pressed.

Figure 2. Keypad

Number	Row	Column
0	1110	1011
1	0111	0111
2	0111	1011
3	0111	1101
4	1011	0111
5	1011	1011
6	1011	1101
7	1101	0111
8	1101	1011
9	1101	1101
A	0111	1110
B	1011	1110
C	1101	1110
D	1110	1110

2.3 ARDUINO UNO BOARD

Arduino UNO R3 board was used and interfaced with LCD via I2C to display the state change. The board is also interfaced with FPGA via 4 I/O pins through the arduino UNO expansion header. The state machines designed in this project have four states: RESET, SRAM A Input, SRAM B Input and FP MAC Calc. LCD displays the current state.

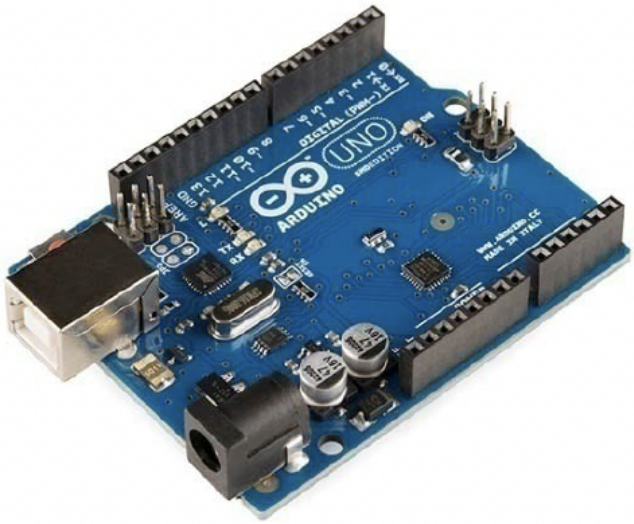

Figure 3. Arduino

2.4 LCD

A 16x2 LCD is used in the project. A 16x2 LCD can display 16 characters per line on each of its two lines. The function of the LCD is to display the current state.


VSS (Ground)
VDD (5V)
VE (Contrast)
RS (Register Select)
RW (Read/Write)
E (Enable)
D0
D1
D2
D3
D4
D5
D6
D7
Backlight Anode
Backlight Cathode

Figure 4. LCD

3. SOFTWARE TOOLS

3.1 QUARTUS PRIME LITE EDITION

Quartus Prime software is used to programme Intel's DE-10 Lite board and the software was specifically developed by Intel's Programmable Solutions Group to facilitate the Programming of various families of the FPGA Boards.

3.2 ARDUINO IDE

The Arduino IDE (Integrated Development Environment) is a software development environment for Arduino. Arduino IDE is used to program an Arduino board to read the current state of the state machine.

4. OVERVIEW OF DESIGN

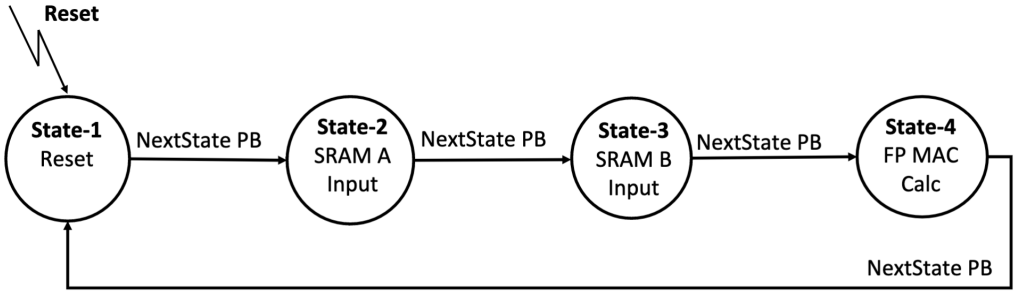

Figure 5. Finite State Diagram

1. The flow of the FP_MAC is represented by the Finite State Diagram. In this FSM, RESET, SRAM A Input, SRAM B Input and FP MAC Calc are the four states that define the complete process.
2. In the state 1, DE-10 lite LCD displays “Reset” when the push button is pressed for the first time.
3. The FSM enters the second stage, when the push button is pressed for the second time, and the LCD displays “SRAM A Input”. It also takes the input from the keypad and displays it on the seven segment display.
4. When the push button is pressed for the third time, the FSM enters the third stage. The LCD displays “SRAM B Input” and also displays the input on the seven segment display.
5. When the push button is pressed for the fourth time, the FSM does the FP MAC Calc by taking the values and displaying it on the seven segment display.
6. When the push button is pressed again, the Finite State Machine enters into the “RESET” stage again.

4.1 BLOCK DIAGRAM OF THE DESIGN

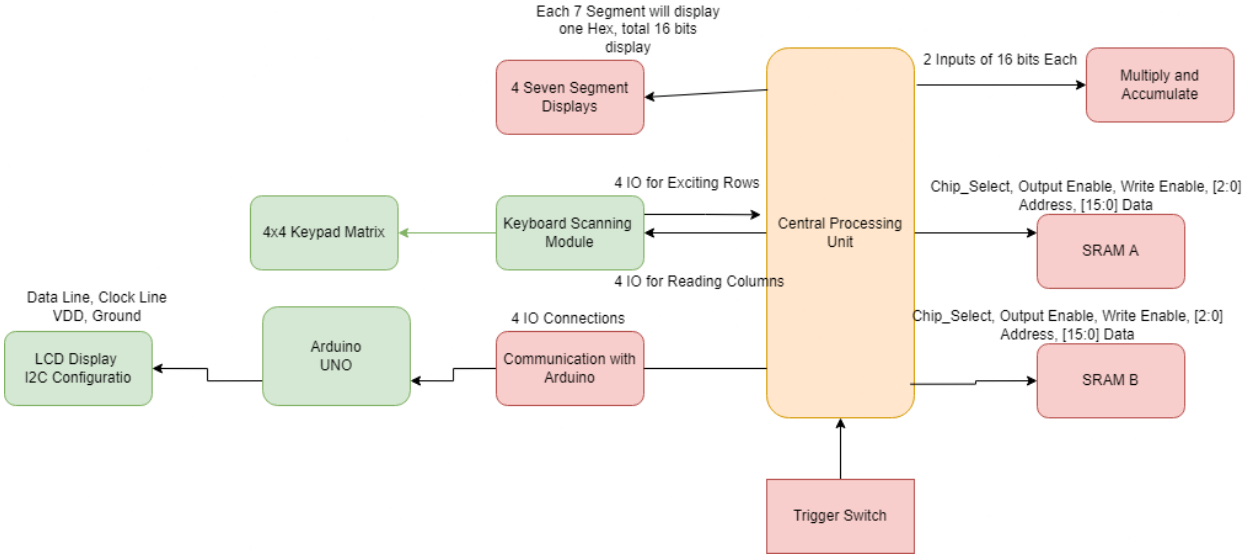

Figure 6. Design Block Diagram

4.2 PIN ASSIGNMENT

arduino_comm[3]	Output	PIN_AA12	4	B4_N0	PIN_AA12	2.5 V		12mA (default)	2 (default)
arduino_comm[2]	Output	PIN_AA11	4	B4_N0	PIN_AA11	2.5 V		12mA (default)	2 (default)
arduino_comm[1]	Output	PIN_Y10	3	B3_N0	PIN_Y10	2.5 V		12mA (default)	2 (default)
arduino_comm[0]	Output	PIN_AB9	3	B3_N0	PIN_AB9	2.5 V		12mA (default)	2 (default)
clk	Input	PIN_P11	3	B3_N0	PIN_P11	2.5 V		12mA (default)	
col_driver[3]	Input	PIN_W5	3	B3_N0	PIN_W5	2.5 V		12mA (default)	
col_driver[2]	Input	PIN_AA14	4	B4_N0	PIN_AA14	2.5 V		12mA (default)	
col_driver[1]	Input	PIN_W12	4	B4_N0	PIN_W12	2.5 V		12mA (default)	
col_driver[0]	Input	PIN_AB12	4	B4_N0	PIN_AB12	2.5 V		12mA (default)	
led[7]	Output	PIN_D15	7	B7_N0	PIN_D15	2.5 V		12mA (default)	2 (default)
led[6]	Output	PIN_C17	7	B7_N0	PIN_C17	2.5 V		12mA (default)	2 (default)
led[5]	Output	PIN_D17	7	B7_N0	PIN_D17	2.5 V		12mA (default)	2 (default)
led[4]	Output	PIN_E16	7	B7_N0	PIN_E16	2.5 V		12mA (default)	2 (default)
led[3]	Output	PIN_C16	7	B7_N0	PIN_C16	2.5 V		12mA (default)	2 (default)
led[2]	Output	PIN_C15	7	B7_N0	PIN_C15	2.5 V		12mA (default)	2 (default)
led[1]	Output	PIN_E15	7	B7_N0	PIN_E15	2.5 V		12mA (default)	2 (default)
led[0]	Output	PIN_C14	7	B7_N0	PIN_C14	2.5 V		12mA (default)	2 (default)
led1[7]	Output	PIN_A19	7	B7_N0	PIN_A19	2.5 V		12mA (default)	2 (default)
led1[6]	Output	PIN_B22	6	B6_N0	PIN_B22	2.5 V		12mA (default)	2 (default)
led1[5]	Output	PIN_C22	6	B6_N0	PIN_C22	2.5 V		12mA (default)	2 (default)
led1[4]	Output	PIN_B21	6	B6_N0	PIN_B21	2.5 V		12mA (default)	2 (default)
led1[3]	Output	PIN_A21	6	B6_N0	PIN_A21	2.5 V		12mA (default)	2 (default)
led1[2]	Output	PIN_B19	7	B7_N0	PIN_B19	2.5 V		12mA (default)	2 (default)
led1[1]	Output	PIN_A20	7	B7_N0	PIN_A20	2.5 V		12mA (default)	2 (default)
led1[0]	Output	PIN_B20	6	B6_N0	PIN_B20	2.5 V		12mA (default)	2 (default)
led2[7]	Output	PIN_D22	6	B6_N0	PIN_D22	2.5 V		12mA (default)	2 (default)
led2[6]	Output	PIN_E17	6	B6_N0	PIN_E17	2.5 V		12mA (default)	2 (default)
led2[5]	Output	PIN_D19	6	B6_N0	PIN_D19	2.5 V		12mA (default)	2 (default)
led2[4]	Output	PIN_C20	6	B6_N0	PIN_C20	2.5 V		12mA (default)	2 (default)
led2[3]	Output	PIN_C19	7	B7_N0	PIN_C19	2.5 V		12mA (default)	2 (default)
led2[2]	Output	PIN_E21	6	B6_N0	PIN_E21	2.5 V		12mA (default)	2 (default)
led2[1]	Output	PIN_E22	6	B6_N0	PIN_E22	2.5 V		12mA (default)	2 (default)
led2[0]	Output	PIN_F21	6	B6_N0	PIN_F21	2.5 V		12mA (default)	2 (default)
led3[7]	Output	PIN_F17	6	B6_N0	PIN_F17	2.5 V		12mA (default)	2 (default)
led3[6]	Output	PIN_F20	6	B6_N0	PIN_F20	2.5 V		12mA (default)	2 (default)
led3[5]	Output	PIN_F19	6	B6_N0	PIN_F19	2.5 V		12mA (default)	2 (default)
led3[4]	Output	PIN_H19	6	B6_N0	PIN_H19	2.5 V		12mA (default)	2 (default)
led3[3]	Output	PIN_J18	6	B6_N0	PIN_J18	2.5 V		12mA (default)	2 (default)

led3[3]	Output	PIN_J18	6	B6_N0	PIN_J18	2.5 V		12mA (default)	2 (default)		
led3[2]	Output	PIN_E19	6	B6_N0	PIN_E19	2.5 V		12mA (default)	2 (default)		
led3[1]	Output	PIN_E20	6	B6_N0	PIN_E20	2.5 V		12mA (default)	2 (default)		
led3[0]	Output	PIN_F18	6	B6_N0	PIN_F18	2.5 V		12mA (default)	2 (default)		
next_trigger	Input	PIN_C10	7	B7_N0	PIN_C10	2.5 V		12mA (default)			
row_driver[3]	Output	PIN_AB11	4	B4_N0	PIN_AB11	2.5 V		12mA (default)	2 (default)		
row_driver[2]	Output	PIN_AB10	4	B4_N0	PIN_AB10	2.5 V		12mA (default)	2 (default)		
row_driver[1]	Output	PIN_AA9	3	B3_N0	PIN_AA9	2.5 V		12mA (default)	2 (default)		
row_driver[0]	Output	PIN_AA8	3	B3_N0	PIN_AA8	2.5 V		12mA (default)	2 (default)		
rst	Input	PIN_C11	7	B7_N0	PIN_C11	2.5 V		12mA (default)			
tracker[7]	Output	PIN_D14	7	B7_N0	PIN_D14	2.5 V		12mA (default)	2 (default)		
tracker[6]	Output	PIN_E14	7	B7_N0	PIN_E14	2.5 V		12mA (default)	2 (default)		
tracker[5]	Output	PIN_C13	7	B7_N0	PIN_C13	2.5 V		12mA (default)	2 (default)		
tracker[4]	Output	PIN_D13	7	B7_N0	PIN_D13	2.5 V		12mA (default)	2 (default)		
tracker[3]	Output	PIN_B10	7	B7_N0	PIN_B10	2.5 V		12mA (default)	2 (default)		
tracker[2]	Output	PIN_A10	7	B7_N0	PIN_A10	2.5 V		12mA (default)	2 (default)		
tracker[1]	Output	PIN_A9	7	B7_N0	PIN_A9	2.5 V		12mA (default)	2 (default)		
tracker[0]	Output	PIN_A8	7	B7_N0	PIN_A8	2.5 V		12mA (default)	2 (default)		
<<new node>>											

Figure 7. Pin Assignment

5. WORKING

A modular approach was used, wherein multiple sub-modules were developed and then instantiated in the main file which helped in debugging and testing. The modules used to develop this project are depicted and described in detail in this section.

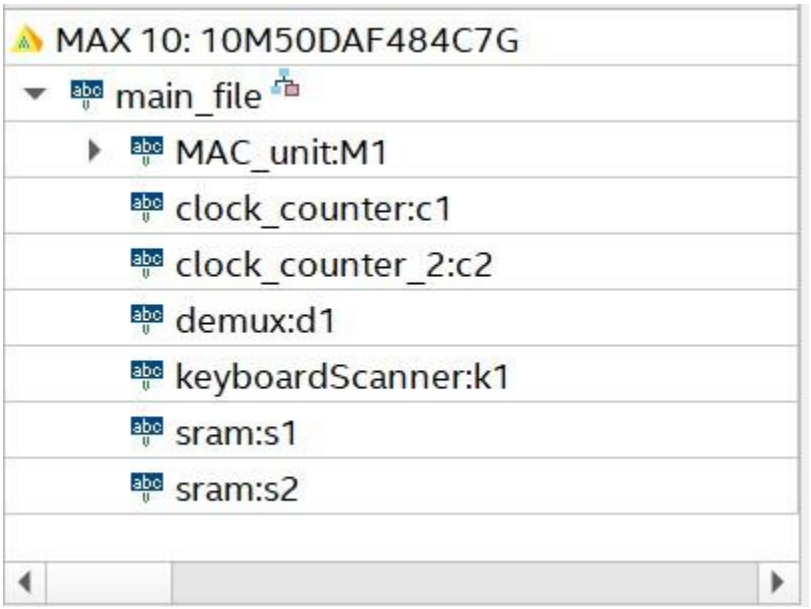

Figure 8 Top Down View of the Project Modules

5. 1 Clock Counter (c1) and Clock Counter 2 (c2)

Two clock counters were used in this project. The first clock counter c1 was used in reading the input from the keyboard by routinely exciting the rows of the 4x4 keypad. The original frequency of the FPGA module of 50MHz was divided by 6553600 to get a resulting frequency of 7.6Hz and the second clock counter was used for powering the FSM module and subsequently which also controls the write and read speed of the FPGA. The second clock divides the 50MHz clock by 24'hFFFFFF which gives a resulting clock of 3Hz.

5.2 Demux (d1)

Demux is a module used by the FSM to interact with the two SRAM's. The module takes a single input and based on its value either the chip select is made low. The Boolean truth table is tabulated below.

Table 1 - Boolean Truth Table of Demux

Select	Chip Select for SRAM A	Chip Select for SRAM B
0	0	1
1	1	0

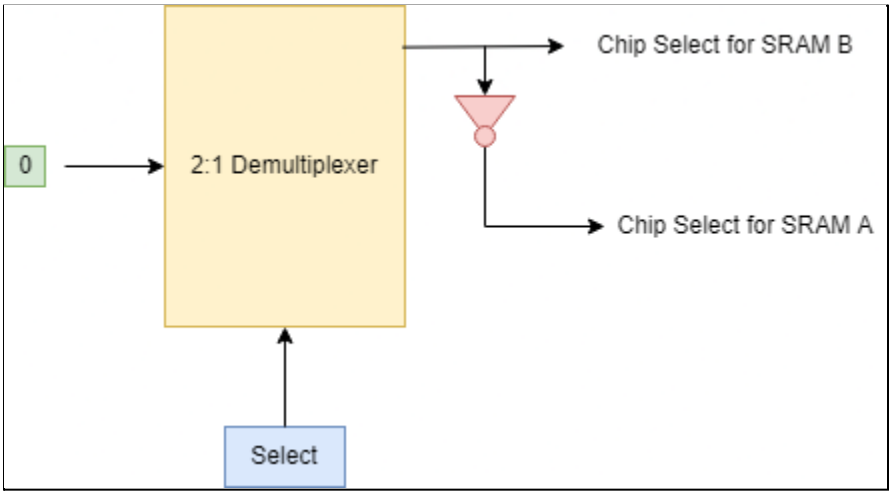

Figure 9 Pictorial Representation of Demultiplexer

5.3 SRAM (s1 and s2)

Static RAM is a module that has been instantiated twice to hold the 16 inputs entered via the keypad, and are called SRAM A and SRAM B. The SRAM module holds 8, half-word values in a register and supports asynchronous read and write operations to it, and uses active-low logic to enable the SRAM. The module uses a 3-bit address to access a memory element and 3 control signals namely chip

select, write enable and output enable which enable the SRAM, enable writing and reading respectively.

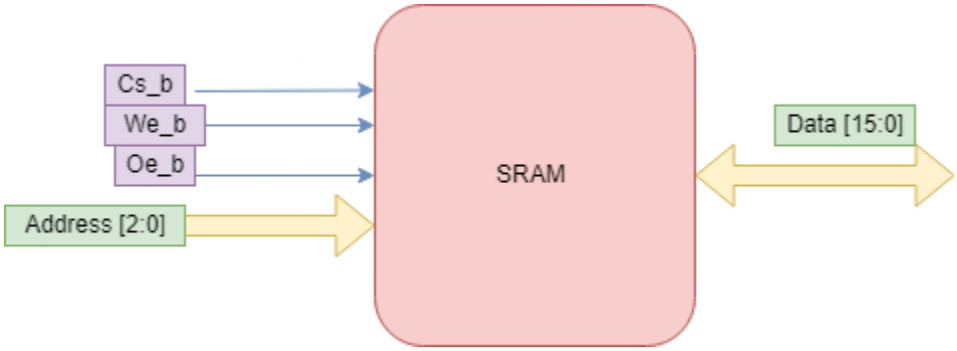

Figure 11 SRAM Model

5.4 MAC Unit (M1)

The MAC unit is a two-stage pipelined unit which takes two half-words as an input and multiplies the two to generate a 32-bit number which is then normalized and treated for overflow/underflow and subsequently is added to the accumulator, and to ensure that the accumulator starts at 0, a ‘MAC RST’ signal was added which is an active high signal.

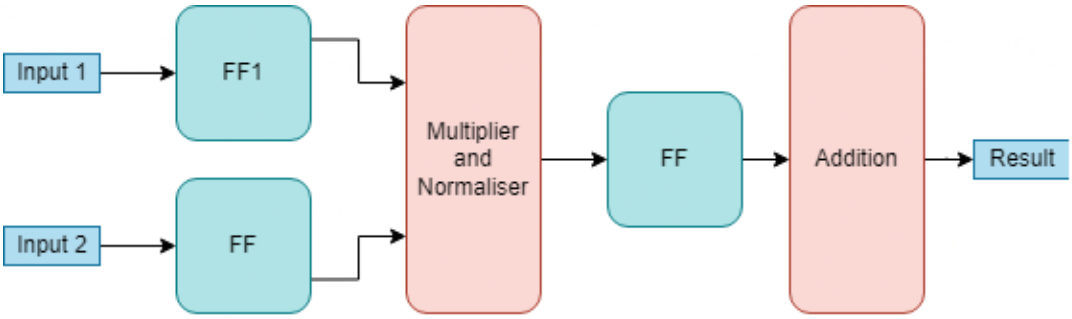

Figure 12 MAC Unit

5.5 Keyboard Scanner

To routinely scan through the 4 rows, a FSM was implemented which periodically grounds one row and reads the inputs, if a switch is pressed, one bit would be 0 and the rest will be 1.

5.6 LCD

The LCD module is hosted on the Arduino and communicates with it using I2C instead of SPI since the former drastically reduces the total number of connections to 4 instead of 8. I2C uses serial communication at a baud rate of 400KHz and based on the input voltages at 4 IO pins, different messages are displayed on the screen.

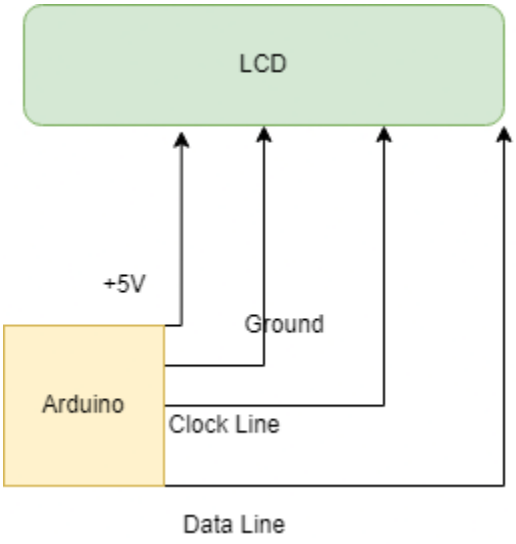

Figure 13 Arduino - LCD Communication via I2C

6. SIMULATION RESULTS

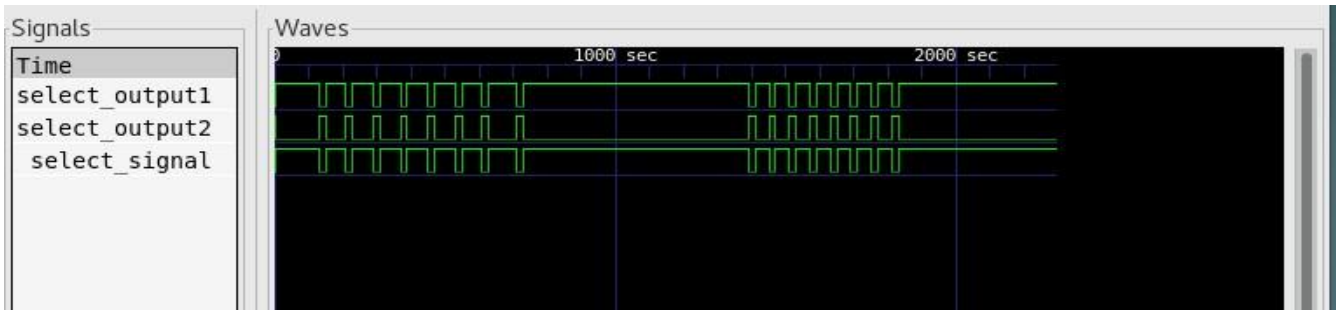

Figure 14 Demux Simulation Output

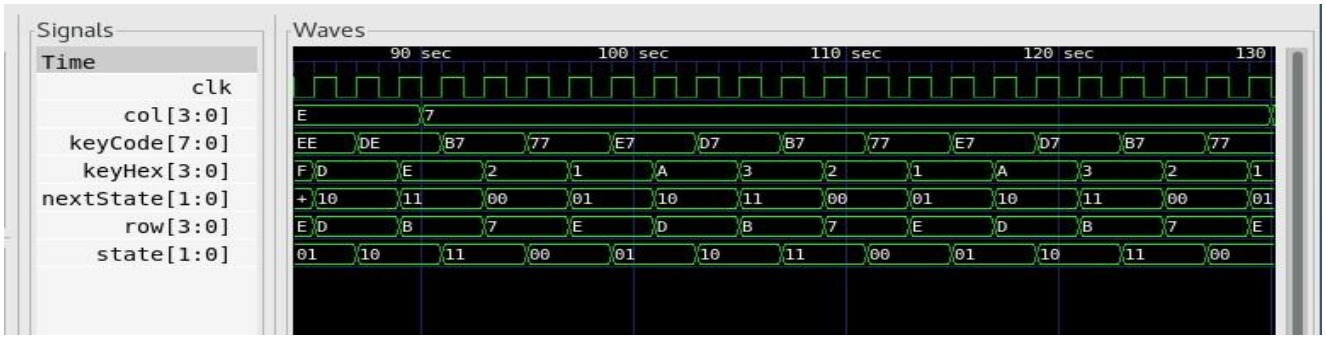

Figure 15 Keyboard Module Routinely Scanning the Input for Data

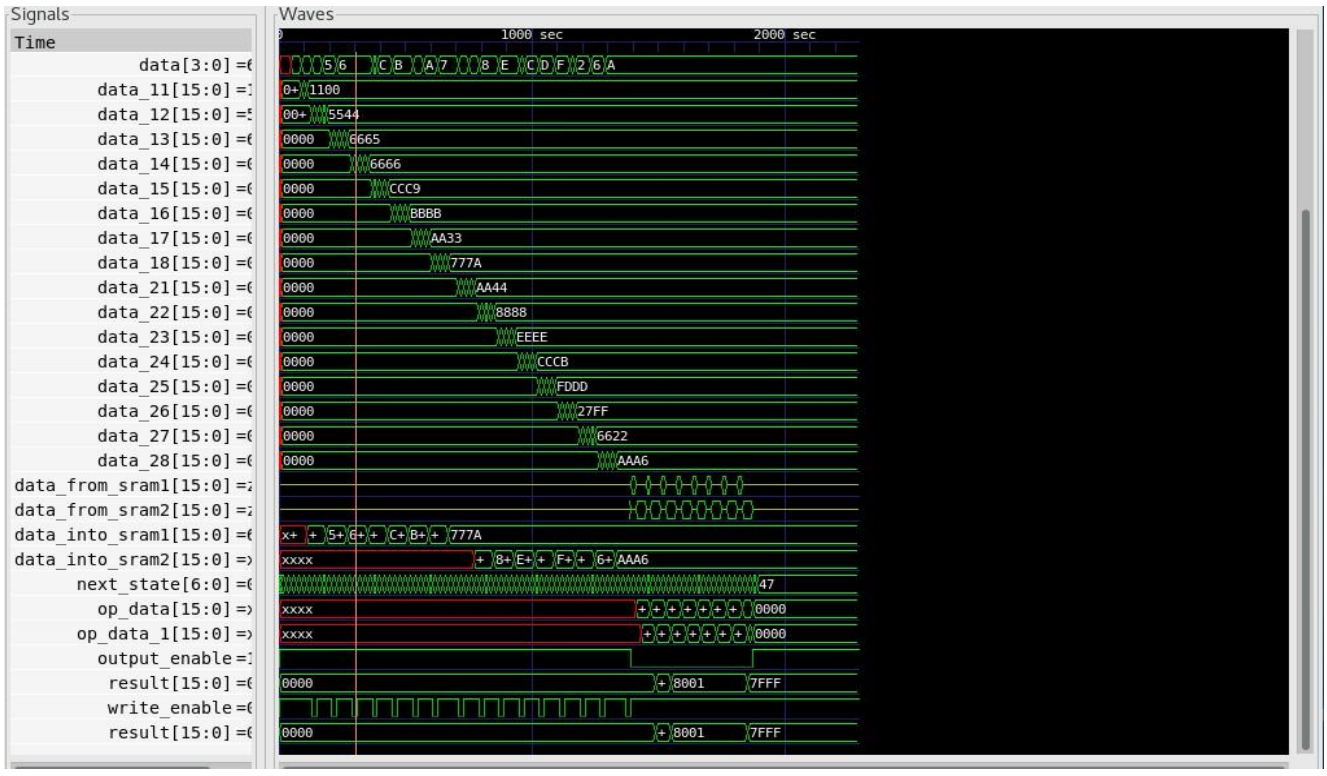

Figure 16 FSM Outputs for Inputs shown in Video

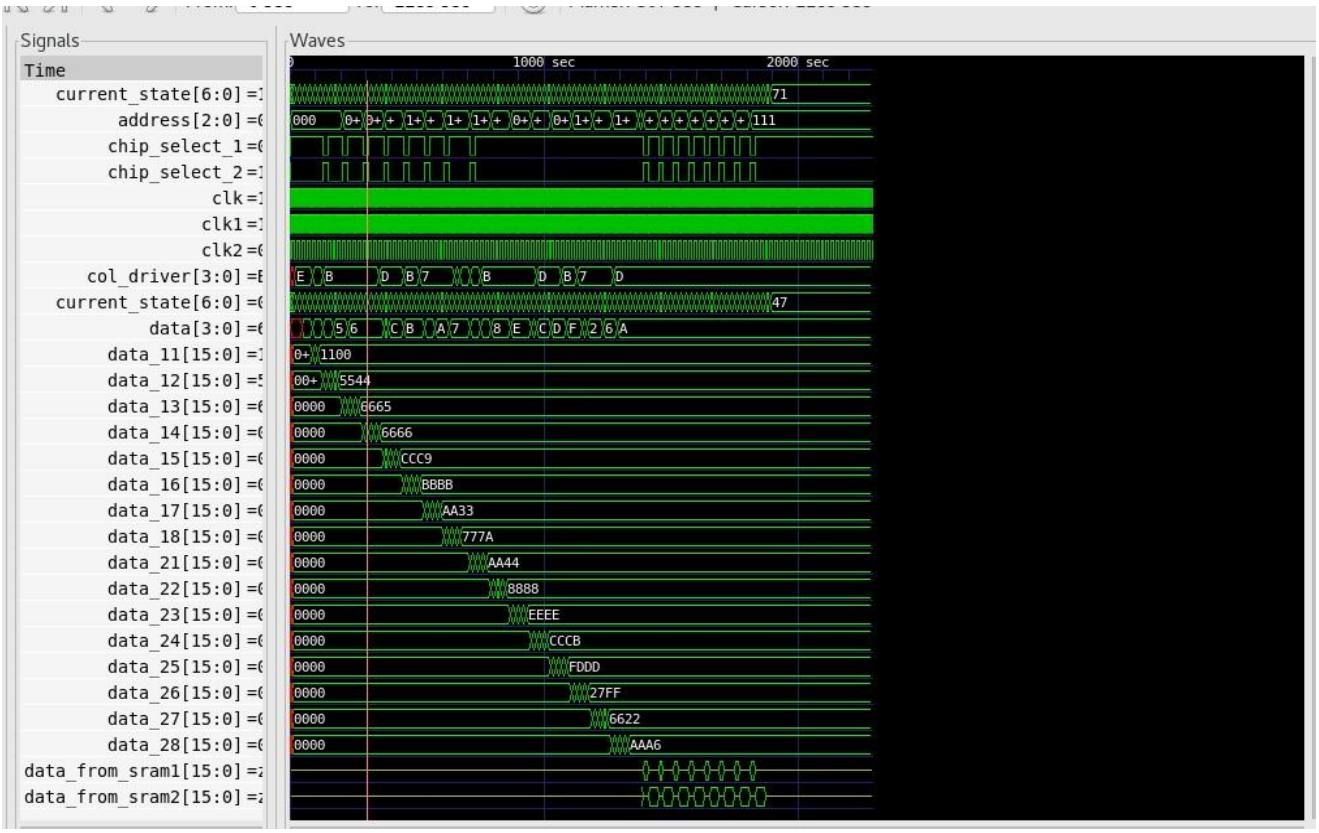

Figure 17 FSM Outputs for Inputs Shown in Video

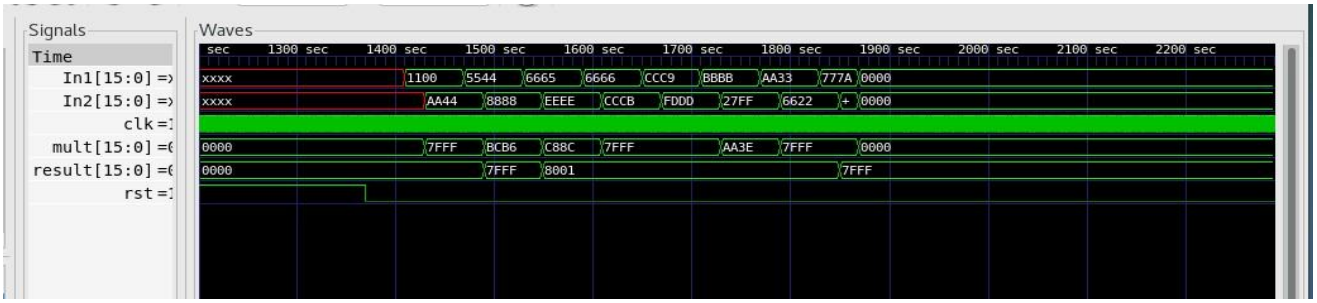

Figure 18 MAC Outputs for Inputs shown in Video

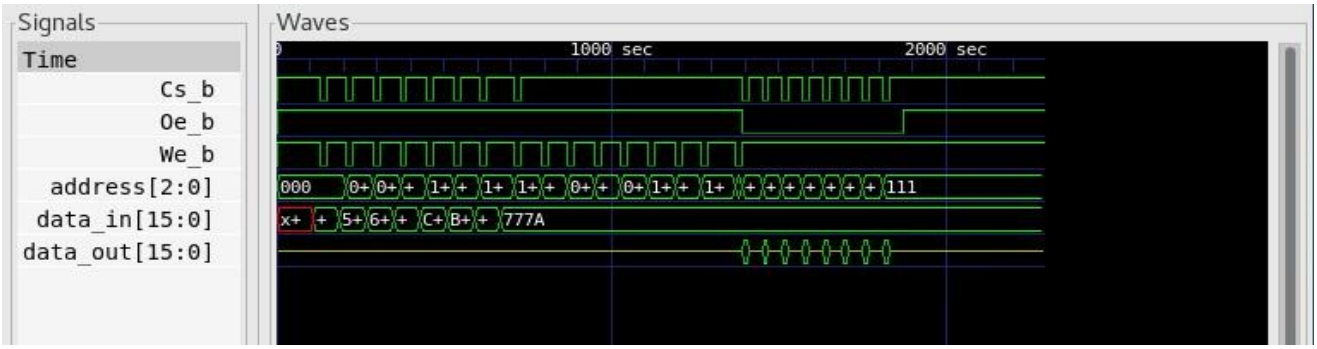

Figure 19 SRAM A Outputs for Inputs shown in Video

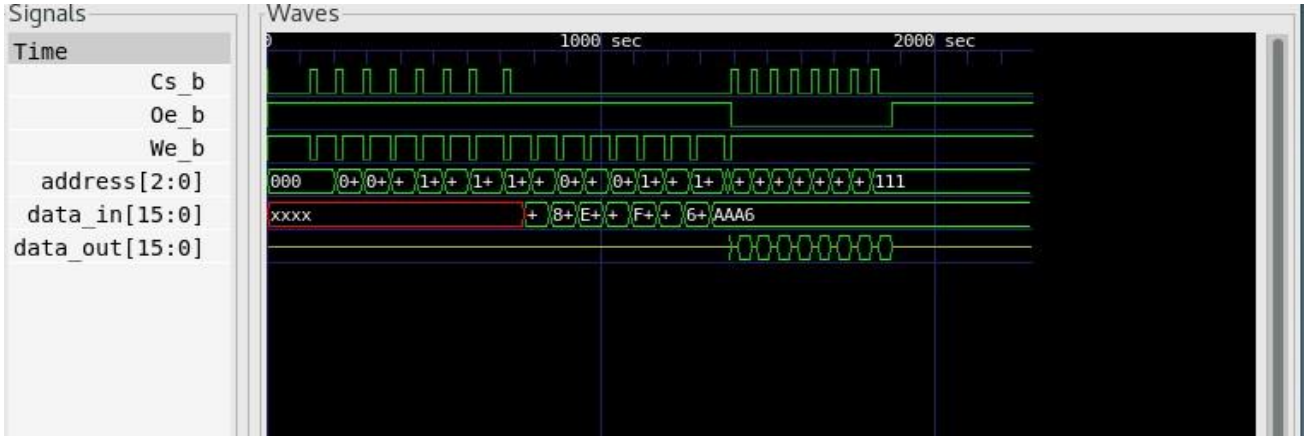

Figure 20 SRAM B Outputs for Inputs shown in Video

7. CONCLUSION

By using a modular approach to design several modules for this project ranging from a SRAM model and a two stage pipelined MAC unit which can handle signed multiplication and addition, the inputs and final results are depicted on the built-in 7 segment display in the FPGA board, while the Arduino board uses a LCD to display the current state of the FPGA board and the two boards communicate via a series of connections and based on their excitation pattern, different messages are displayed on the screen.

8. ANNEXURE

8.1 Main Module

```
module main_file(input clk,rst, next_trigger,input wire[3:0] col_driver, output reg [3:0] arduino_comm, output reg [7:0] tracker, output wire [3:0] row_driver, output reg [7:0] led,led1,led2,led3);

parameter S0 = 7'h0;

parameter S1 = 7'h1;

parameter S2 = 7'h2;

parameter S3 = 7'h3;

parameter S4 = 7'h04;

parameter S5 = 7'h5;

parameter S6 = 7'h6;

parameter S7 = 7'h7;

parameter S8 = 7'h8;

parameter S9 = 7'h9;

parameter S10 = 7'ha;

parameter S11 = 7'hb;

parameter S12 = 7'hc;

parameter S13 = 7'hd;

parameter S14 = 7'he;
```

```
parameter S15 = 7'hf;
parameter S16 = 7'h10;
parameter S17 = 7'h11;
parameter S18 = 7'h12;
parameter S19 = 7'h13;
parameter S20 = 7'h14;
parameter S21 = 7'h15;
parameter S22 = 7'h16;
parameter S23 = 7'h17;
parameter S24 = 7'h18;
parameter S25 = 7'h19;
parameter S26 = 7'h1a;
parameter S27 = 7'h1b;
parameter S28 = 7'h1c;
parameter S29 = 7'h1d;
parameter S30 = 7'h1e;
parameter S31 = 7'h1f;
parameter S32 = 7'h20;
parameter S33 = 7'h21;
parameter S34 = 7'h22;
parameter S35 = 7'h23;
```

```
parameter S36 = 7'h24;
parameter S37 = 7'h25;
parameter S38 = 7'h26;
parameter S39 = 7'h27;
parameter S40 = 7'h28;
parameter S41 = 7'h29;
parameter S42 = 7'h2a;
parameter S43 = 7'h2b;
parameter S44 = 7'h2c;
parameter S45 = 7'h2d;
parameter S46 = 7'h2e;
parameter S47 = 7'h2f;
parameter S48 = 7'h30;
parameter S49 = 7'h31;
parameter S50 = 7'h32;
parameter S51 = 7'h33;
parameter S52 = 7'h34;
parameter S53 = 7'h35;
parameter S54  = 7'h36;
parameter S55 = 7'h37;
parameter S56 = 7'h38;
```

```
parameter S57 = 7'h39;

parameter S58 = 7'h3a;

parameter S59  = 7'h3b;

parameter S60 = 7'h3c;

parameter S61 = 7'h3d;

parameter S62 = 7'h3e;

parameter S63  =7'h3f;

parameter S64 = 7'h40;

parameter S65 = 7'h41;

parameter S66 = 7'h42;

parameter S67 = 7'h43;

parameter S68 = 7'h44;

parameter S69 = 7'h45;

parameter S70 = 7'h46;

parameter S71 = 7'h47;

parameter S72 = 7'h48;

parameter S73 = 7'h49;

parameter S74 = 7'h4A;

parameter S75 = 7'h4B;

parameter S76 = 7'h4C;

parameter S77 = 7'h4D;
```

```
parameter S78 = 7'h4E;

parameter S79 = 7'h4F;

parameter S80 = 7'h50;

parameter S81 = 7'h51;

parameter S82 = 7'h52;

parameter S83 = 7'h53;



reg [2:0] address;

reg [6:0] current_state,next_state = 0;

reg [15:0] data_11,data_12,data_13,data_14,data_15,data_16,data_17,data_18;

reg [15:0] data_21,data_22,data_23,data_24,data_25,data_26,data_27,data_28;

reg [15:0] data_into_sram1,data_into_sram2;

reg write_enable,output_enable,select,mac_rst;

reg [15:0] op_data,op_data_1;

wire [15:0] result;

wire [3:0] data;

wire clk1,clk2;


wire [15:0] data_from_sram1,data_from_sram2;
```

```
wire chip_select_1,chip_select_2 ;
initial begin
        write_enable = 1'b1;
        select = 1'b1;
        output_enable = 1'b1;
        current_state = S0;
        next_state = S0;
        mac_rst = 1;
        address = 0;
end

clock_counter c1(clk,clk1);
clock_counter_2 c2(clk,clk2);

keyboardScanner k1(clk1,col_driver, row_driver,data);
demux d1(select,chip_select_1,chip_select_2);



MAC_unit M1(clk1,mac_rst,op_data,op_data_1,result);
sram
s1(chip_select_1,write_enable,output_enable,address,data_into_sram1,data_from_
sram1);
```

```
sram
s2(chip_select_2,write_enable,output_enable,address,data_into_sram2,data_from_
sram2);

integer i =0;

always @(*) begin

        case(current_state)

        S0: begin

                data_11 = 0;

                data_12 = 0;

                data_13 = 0;

                data_14 = 0;

                data_15 = 0;

                data_16 = 0;

                data_17 = 0;

                data_18 = 0;

                data_21 =0;

                data_22 = 0;

                data_23 = 0;

                data_24 = 0;

                data_25 =0;

                data_26= 0;

                data_27 = 0;
```

```
        data_28 =0;
        arduino_comm = 4'h1;
        next_state = S74;
        seven_segment_display(4'h1,led);
        seven_segment_display(4'h7,led1);
        seven_segment_display(4'h2,led2);
        seven_segment_display(4'hE,led3);
end
S1: begin
        data_11[15:4] = 0;
        data_11[3:0] = data;
        seven_segment_display(data_11[3:0],led);
        seven_segment_display(data_11[7:4],led1);
        seven_segment_display(data_11[11:8],led2);
        seven_segment_display(data_11[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S2;
        tracker = 8'hFE;
end
S2: begin
        data_11[15:8] = 0;
```

```
        data_11[7:4] = data;
        seven_segment_display(data_11[3:0],led);
        seven_segment_display(data_11[7:4],led1);
        seven_segment_display(data_11[11:8],led2);
        seven_segment_display(data_11[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S3;
        tracker = 8'hFE;
end
S3: begin
        data_11[15:12] = 0;
        data_11[11:8] = data;
        seven_segment_display(data_11[3:0],led);
        seven_segment_display(data_11[7:4],led1);
        seven_segment_display(data_11[11:8],led2);
        seven_segment_display(data_11[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S4;
        tracker = 8'hFE;
end
S4: begin
```

```
        data_11[15:12] = data;
        seven_segment_display(data_11[3:0],led);
        seven_segment_display(data_11[7:4],led1);
        seven_segment_display(data_11[11:8],led2);
        seven_segment_display(data_11[15:12],led3);
        arduino_comm = 4'h2;
        write_to_memory_1(data_11);
        next_state = S5;
        tracker = 8'hFE;
end
S5:begin
        write_high_1(3'b000);
        data_12[15:4] = 0;
        data_12[3:0] = data;
        seven_segment_display(data_12[3:0],led);
        seven_segment_display(data_12[7:4],led1);
        seven_segment_display(data_12[11:8],led2);
        seven_segment_display(data_12[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S6;
        tracker = 8'hFD;
```

```
end
S6: begin
        close_write;
        data_12[15:8] = 0;
        data_12[7:4] = data;
        seven_segment_display(data_12[3:0],led);
        seven_segment_display(data_12[7:4],led1);
        seven_segment_display(data_12[11:8],led2);
        seven_segment_display(data_12[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S7;
        tracker = 8'hFD;
end
S7: begin
        data_12[15:12] = 0;
        data_12[11:8] = data;
        seven_segment_display(data_12[3:0],led);
        seven_segment_display(data_12[7:4],led1);
        seven_segment_display(data_12[11:8],led2);
        seven_segment_display(data_12[15:12],led3);
        arduino_comm = 4'h2;
```

```
        next_state = S8;
        tracker = 8'hFD;
end
S8: begin
        data_12[15:12] = data;
        seven_segment_display(data_12[3:0],led);
        seven_segment_display(data_12[7:4],led1);
        seven_segment_display(data_12[11:8],led2);
        seven_segment_display(data_12[15:12],led3);
        write_to_memory_1(data_12);
        arduino_comm = 4'h2;
        next_state = S9;
        tracker = 8'hFD;
end
S9: begin
        write_high_1(3'b001);
        data_13[15:4] = 0;
        data_13[3:0] = data;
        seven_segment_display(data_13[3:0],led);
        seven_segment_display(data_13[7:4],led1);
        seven_segment_display(data_13[11:8],led2);
```

```
        seven_segment_display(data_13[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S10;
        tracker = 8'hFB;
end
S10: begin
        close_write;
        data_13[15:8] = 0;
        data_13[7:4] = data;
        seven_segment_display(data_13[3:0],led);
        seven_segment_display(data_13[7:4],led1);
        seven_segment_display(data_13[11:8],led2);
        seven_segment_display(data_13[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S11;
        tracker = 8'hFB;
end
S11: begin
        data_13[15:12] = 0;
        data_13[11:8] = data;
        seven_segment_display(data_13[3:0],led);
```

```
        seven_segment_display(data_13[7:4],led1);
        seven_segment_display(data_13[11:8],led2);
        seven_segment_display(data_13[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S12;
        tracker = 8'hFB;
end
S12: begin
        data_13[15:12] = data;
        seven_segment_display(data_13[3:0],led);
        seven_segment_display(data_13[7:4],led1);
        seven_segment_display(data_13[11:8],led2);
        seven_segment_display(data_13[15:12],led3);
        arduino_comm = 4'h2;
        write_to_memory_1(data_13);
        next_state = S13;
        tracker = 8'hFB;
end
S13:begin
        write_high_1(3'b010);
        data_14[15:4] = 0;
```

```
        data_14[3:0] = data;
        seven_segment_display(data_14[3:0],led);
        seven_segment_display(data_14[7:4],led1);
        seven_segment_display(data_14[11:8],led2);
        seven_segment_display(data_14[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S14;
        tracker = 8'hF7;
end
S14: begin
        close_write;
        data_14[15:8] = 0;
        data_14[7:4] = data;
        seven_segment_display(data_14[3:0],led);
        seven_segment_display(data_14[7:4],led1);
        seven_segment_display(data_14[11:8],led2);
        seven_segment_display(data_14[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S15;
        tracker = 8'hF7;
end
```

```
S15: begin
        data_14[15:12] = 0;
        data_14[11:8] = data;
        seven_segment_display(data_14[3:0],led);
        seven_segment_display(data_14[7:4],led1);
        seven_segment_display(data_14[11:8],led2);
        seven_segment_display(data_14[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S16;
        tracker = 8'hF7;
end
S16: begin
        data_14[15:12] = data;
        seven_segment_display(data_14[3:0],led);
        seven_segment_display(data_14[7:4],led1);
        seven_segment_display(data_14[11:8],led2);
        seven_segment_display(data_14[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S17;
        tracker = 8'hF7;
        write_to_memory_1(data_14);
```

```
end
S17: begin
        write_high_1(3'b011);

        data_15[15:4] = 0;
        data_15[3:0] = data;
        seven_segment_display(data_15[3:0],led);
        seven_segment_display(data_15[7:4],led1);
        seven_segment_display(data_15[11:8],led2);
        seven_segment_display(data_15[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S18;
        tracker = 8'hEF;
end
S18: begin
        close_write;
        data_15[15:8] = 0;
        data_15[7:4] = data;
        seven_segment_display(data_15[3:0],led);
        seven_segment_display(data_15[7:4],led1);
        seven_segment_display(data_15[11:8],led2);
```

```
        seven_segment_display(data_15[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S19;
        tracker = 8'hF7;
end
S19: begin
        data_15[15:12] = 0;
        data_15[11:8] = data;
        seven_segment_display(data_15[3:0],led);
        seven_segment_display(data_15[7:4],led1);
        seven_segment_display(data_15[11:8],led2);
        seven_segment_display(data_15[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S20;
        tracker = 8'hF7;

end
S20: begin
        data_15[15:12] = data;
        seven_segment_display(data_15[3:0],led);
        seven_segment_display(data_15[7:4],led1);
```

```
        seven_segment_display(data_15[11:8],led2);
        seven_segment_display(data_15[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S21;
        write_to_memory_1(data_15);
        tracker = 8'hF7;
end
S21:begin
        write_high_1(3'b100);
        data_16[15:4] = 0;
        data_16[3:0] = data;
        seven_segment_display(data_16[3:0],led);
        seven_segment_display(data_16[7:4],led1);
        seven_segment_display(data_16[11:8],led2);
        seven_segment_display(data_16[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S22;
        tracker = 8'hF7;
end
S22: begin
        close_write;
```

```
        data_16[15:8] = 0;
        data_16[7:4] = data;
        seven_segment_display(data_16[3:0],led);
        seven_segment_display(data_16[7:4],led1);
        seven_segment_display(data_16[11:8],led2);
        seven_segment_display(data_16[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S23;
        tracker = 8'hDF;
end
S23: begin
        data_16[15:12] = 0;
        data_16[11:8] = data;
        seven_segment_display(data_16[3:0],led);
        seven_segment_display(data_16[7:4],led1);
        seven_segment_display(data_16[11:8],led2);
        seven_segment_display(data_16[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S24;
        tracker = 8'hDF;
end
```

```
S24: begin
        data_16[15:12] = data;
        seven_segment_display(data_16[3:0],led);
        seven_segment_display(data_16[7:4],led1);
        seven_segment_display(data_16[11:8],led2);
        seven_segment_display(data_16[15:12],led3);

        write_to_memory_1(data_16);
        arduino_comm = 4'h2;
        next_state = S25;
        tracker = 8'hDF;
end
S25: begin
        write_high_1(3'b101);
        data_17[15:4] = 0;
        data_17[3:0] = data;
        seven_segment_display(data_17[3:0],led);
        seven_segment_display(data_17[7:4],led1);
        seven_segment_display(data_17[11:8],led2);
        seven_segment_display(data_17[15:12],led3);
        arduino_comm = 4'h2;
```

```
        next_state = S26;
        tracker = 8'hBF;
end
S26: begin
        close_write;
        data_17[15:8] = 0;
        data_17[7:4] = data;
        seven_segment_display(data_17[3:0],led);
        seven_segment_display(data_17[7:4],led1);
        seven_segment_display(data_17[11:8],led2);
        seven_segment_display(data_17[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S27;
        tracker = 8'hBF;
end
S27: begin
        data_17[15:12]  = 0;
        data_17[11:8] = data;
        seven_segment_display(data_17[3:0],led);
        seven_segment_display(data_17[7:4],led1);
        seven_segment_display(data_17[11:8],led2);
```

```
        seven_segment_display(data_17[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S28;
        tracker = 8'hBF;
end
S28:begin
        data_17[15:12] = data;
        seven_segment_display(data_17[3:0],led);
        seven_segment_display(data_17[7:4],led1);
        seven_segment_display(data_17[11:8],led2);
        seven_segment_display(data_17[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S29;
        tracker = 8'hBF;
        write_to_memory_1(data_17);
end
S29: begin
        write_high_1(3'b110);
        data_18[15:4] = 0;
        data_18[3:0] = data;
        seven_segment_display(data_18[3:0],led);
```

```
        seven_segment_display(data_18[7:4],led1);
        seven_segment_display(data_18[11:8],led2);
        seven_segment_display(data_18[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S30;
        tracker = 8'h7F;
end
S30: begin
        close_write;
        data_18[15:8] = 0;
        data_18[7:4] = data;
        seven_segment_display(data_18[3:0],led);
        seven_segment_display(data_18[7:4],led1);
        seven_segment_display(data_18[11:8],led2);
        seven_segment_display(data_18[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S31;
        tracker = 8'h7F;
end
S31: begin
        data_18[15:12] = 0;
```

```
        data_18[11:8] = data;
        seven_segment_display(data_18[3:0],led);
        seven_segment_display(data_18[7:4],led1);
        seven_segment_display(data_18[11:8],led2);
        seven_segment_display(data_18[15:12],led3);
        arduino_comm = 4'h2;
        next_state = S32;
        tracker = 8'h7F;
end
S32: begin
        data_18[15:12] = data;
        seven_segment_display(data_18[3:0],led);
        seven_segment_display(data_18[7:4],led1);
        seven_segment_display(data_18[11:8],led2);
        seven_segment_display(data_18[15:12],led3);

        write_to_memory_1(data_18);
        arduino_comm = 4'h2;
        next_state = S72;
        tracker = 8'h7F;
end
```

```
S33: begin
        write_high_1(3'b111);
        arduino_comm = 4'h4;
        data_21[15:4] = 0;
        data_21[3:0] = data;
        seven_segment_display(data_21[3:0],led);
        seven_segment_display(data_21[7:4],led1);
        seven_segment_display(data_21[11:8],led2);
        seven_segment_display(data_21[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S34;
        tracker = 8'hFE;
end
S34: begin
        close_write;
        arduino_comm = 4'h4;
        data_21[15:8] = 0;
        data_21[7:4] = data;
        seven_segment_display(data_21[3:0],led);
        seven_segment_display(data_21[7:4],led1);
        seven_segment_display(data_21[11:8],led2);
```

```
        seven_segment_display(data_21[15:12],led3);
        next_state = S35;
        tracker = 8'hFE;
end
S35: begin
        data_21[15:12] = 0;
        data_21[11:8] = data;
        seven_segment_display(data_21[3:0],led);
        seven_segment_display(data_21[7:4],led1);
        seven_segment_display(data_21[11:8],led2);
        seven_segment_display(data_21[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S36;
        tracker = 8'hFE;
end
S36:begin
        data_21[15:12] = data;
        seven_segment_display(data_21[3:0],led);
        seven_segment_display(data_21[7:4],led1);
        seven_segment_display(data_21[11:8],led2);
        seven_segment_display(data_21[15:12],led3);
```

```
        next_state = S37;
        arduino_comm = 4'h4;
        write_to_memory_2(data_21);
        tracker = 8'hFE;
end
S37: begin
        write_high_2(3'b000);
        data_22[15:4] = 0;
        data_22[3:0] = data;
        seven_segment_display(data_22[3:0],led);
        seven_segment_display(data_22[7:4],led1);
        seven_segment_display(data_22[11:8],led2);
        seven_segment_display(data_22[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S38;
        tracker = 8'hFD;
end
S38: begin
        close_write;
        data_22[15:8] = 0;
        data_22[7:4] = data;
```

```
        seven_segment_display(data_22[3:0],led);
        seven_segment_display(data_22[7:4],led1);
        seven_segment_display(data_22[11:8],led2);
        seven_segment_display(data_22[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S39;
        tracker = 8'hFD;
end
S39: begin
        data_22[15:12] = 0;
        data_22[11:8] = data;
        seven_segment_display(data_22[3:0],led);
        seven_segment_display(data_22[7:4],led1);
        seven_segment_display(data_22[11:8],led2);
        seven_segment_display(data_22[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S40;
        tracker = 8'hFD;
end
S40: begin
        data_22[15:12] = data;
```

```
        seven_segment_display(data_22[3:0],led);
        seven_segment_display(data_22[7:4],led1);
        seven_segment_display(data_22[11:8],led2);
        seven_segment_display(data_22[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S41;

        write_to_memory_2(data_22);
        tracker = 8'hFD;
end
S41: begin
        write_high_2(3'b001);
        data_23[15:4] = 0;
        data_23[3:0] = data;
        seven_segment_display(data_23[3:0],led);
        seven_segment_display(data_23[7:4],led1);
        seven_segment_display(data_23[11:8],led2);
        seven_segment_display(data_23[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S42;
        tracker = 8'hFB;
```

```
end
S42: begin
        close_write;
        data_23[15:8] = 0;
        data_23[7:4] = data;
        seven_segment_display(data_23[3:0],led);
        seven_segment_display(data_23[7:4],led1);
        seven_segment_display(data_23[11:8],led2);
        seven_segment_display(data_23[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S43;
        tracker = 8'hFB;
end
S43: begin
        data_23[15:12] = 0;
        data_23[11:8] = data;
        seven_segment_display(data_23[3:0],led);
        seven_segment_display(data_23[7:4],led1);
        seven_segment_display(data_23[11:8],led2);
        seven_segment_display(data_23[15:12],led3);
        arduino_comm = 4'h4;
```

```
        next_state = S44;
        tracker = 8'hFB;
end
S44:begin
        data_23[15:12] = data;
        seven_segment_display(data_23[3:0],led);
        seven_segment_display(data_23[7:4],led1);
        seven_segment_display(data_23[11:8],led2);
        seven_segment_display(data_23[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S45;
        write_to_memory_2(data_23);

        tracker = 8'hFB;
end
S45: begin
        write_high_2(3'b010);
        data_24[15:4] = 0;
        data_24[3:0] = data;
        seven_segment_display(data_24[3:0],led);
        seven_segment_display(data_24[7:4],led1);
```

```
        seven_segment_display(data_24[11:8],led2);
        seven_segment_display(data_24[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S46;
        tracker = 8'hF7;
end
S46: begin
        close_write;
        data_24[15:8] = 0;
        data_24[7:4] = data;
        seven_segment_display(data_24[3:0],led);
        seven_segment_display(data_24[7:4],led1);
        seven_segment_display(data_24[11:8],led2);
        seven_segment_display(data_24[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S47;
        tracker = 8'hF7;
end
S47: begin
        data_24[15:12] = 0;
        data_24[11:8] = data;
```

```
        seven_segment_display(data_24[3:0],led);
        seven_segment_display(data_24[7:4],led1);
        seven_segment_display(data_24[11:8],led2);
        seven_segment_display(data_24[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S48;
        tracker = 8'hF7;
end
S48: begin
        data_24[15:12] = data;
        seven_segment_display(data_24[3:0],led);
        seven_segment_display(data_24[7:4],led1);
        seven_segment_display(data_24[11:8],led2);
        seven_segment_display(data_24[15:12],led3);
        arduino_comm = 4'h4;
        write_to_memory_2(data_24);
        next_state = S49;
        tracker = 8'hF7;
end
S49: begin
        write_high_2(3'b011);
```

```
        data_25[15:4] = 0;
        data_25[3:0] = data;
        seven_segment_display(data_25[3:0],led);
        seven_segment_display(data_25[7:4],led1);
        seven_segment_display(data_25[11:8],led2);
        seven_segment_display(data_25[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S50;
        tracker = 8'hEF;
end
S50: begin
        close_write;
        data_25[15:8] = 0;
        data_25[7:4] = data;
        seven_segment_display(data_25[3:0],led);
        seven_segment_display(data_25[7:4],led1);
        seven_segment_display(data_25[11:8],led2);
        seven_segment_display(data_25[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S51;
        tracker = 8'hEF;
```

```
end
S51: begin
        data_25[15:12] = 0;
        data_25[11:8] = data;
        seven_segment_display(data_25[3:0],led);
        seven_segment_display(data_25[7:4],led1);
        seven_segment_display(data_25[11:8],led2);
        seven_segment_display(data_25[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S52;
        tracker = 8'hEF;
end
S52:begin
        data_25[15:12] = data;
        seven_segment_display(data_25[3:0],led);
        seven_segment_display(data_25[7:4],led1);
        seven_segment_display(data_25[11:8],led2);
        seven_segment_display(data_25[15:12],led3);
        arduino_comm = 4'h4;
        next_state = S53;
        write_to_memory_2(data_25);
```

```
        tracker = 8'hEF;
end
S53: begin
        write_high_2(3'b100);

        data_26[15:4] = 0;
        data_26[3:0] = data;
        seven_segment_display(data_26[3:0],led);
        seven_segment_display(data_26[7:4],led1);
        seven_segment_display(data_26[11:8],led2);
        seven_segment_display(data_26[15:12],led3);
        arduino_comm=4'h4;
        next_state = S54;
        tracker = 8'hDF;
end
S54: begin
        close_write;
        data_26[15:8] = 0;
        data_26[7:4] = data;
        seven_segment_display(data_26[3:0],led);
```

```
        seven_segment_display(data_26[7:4],led1);
        seven_segment_display(data_26[11:8],led2);
        seven_segment_display(data_26[15:12],led3);
        arduino_comm=4'h4;
        next_state = S55;
        tracker = 8'hDF;
end
S55: begin
        data_26[15:12] = 0;
        data_26[11:8] = data;
        seven_segment_display(data_26[3:0],led);
        seven_segment_display(data_26[7:4],led1);
        seven_segment_display(data_26[11:8],led2);
        seven_segment_display(data_26[15:12],led3);
        arduino_comm=4'h4;
        next_state = S56;
        tracker = 8'hDF;
end
S56: begin
        data_26[15:12] = data;
        seven_segment_display(data_26[3:0],led);
```

```
        seven_segment_display(data_26[7:4],led1);
        seven_segment_display(data_26[11:8],led2);
        seven_segment_display(data_26[15:12],led3);
        arduino_comm=4'h4;
        next_state = S57;
        write_to_memory_2(data_26);
        tracker = 8'hDF;
end
S57: begin
        write_high_2(3'b101);
        data_27[15:4] = 0;
        data_27[3:0] = data;
        seven_segment_display(data_27[3:0],led);
        seven_segment_display(data_27[7:4],led1);
        seven_segment_display(data_27[11:8],led2);
        seven_segment_display(data_27[15:12],led3);
        arduino_comm=4'h4;
        next_state = S58;
        tracker = 8'hBF;
end
S58: begin
```

```
        close_write;
        data_27[15:8] = 0;
        data_27[7:4] = data;
        seven_segment_display(data_27[3:0],led);
        seven_segment_display(data_27[7:4],led1);
        seven_segment_display(data_27[11:8],led2);
        seven_segment_display(data_27[15:12],led3);
        arduino_comm=4'h4;
        next_state = S59;
        tracker = 8'hBF;
end
S59: begin
        data_27[15:12] = 0;
        data_27[11:8] = data;
        seven_segment_display(data_27[3:0],led);
        seven_segment_display(data_27[7:4],led1);
        seven_segment_display(data_27[11:8],led2);
        seven_segment_display(data_27[15:12],led3);
        arduino_comm=4'h4;
        next_state = S60;
        tracker = 8'hBF;
```

```
end
S60:begin
        data_27[15:12] = data;
        seven_segment_display(data_27[3:0],led);
        seven_segment_display(data_27[7:4],led1);
        seven_segment_display(data_27[11:8],led2);
        seven_segment_display(data_27[15:12],led3);
        arduino_comm=4'h4;
        next_state = S61;
        write_to_memory_2(data_27);
        tracker = 8'hBF;
end
S61: begin
        write_high_2(3'b110);
        data_28[15:4] = 0;
        data_28[3:0] = data;
        seven_segment_display(data_28[3:0],led);
        seven_segment_display(data_28[7:4],led1);
        seven_segment_display(data_28[11:8],led2);
        seven_segment_display(data_28[15:12],led3);
        arduino_comm=4'h4;
```

```
        next_state = S62;
        tracker = 8'h7F;
end
S62: begin
        close_write;
        data_28[15:8] = 0;
        data_28[7:4] = data;
        seven_segment_display(data_28[3:0],led);
        seven_segment_display(data_28[7:4],led1);
        seven_segment_display(data_28[11:8],led2);
        seven_segment_display(data_28[15:12],led3);
        arduino_comm=4'h4;
        next_state = S63;
        tracker = 8'h7F;
end
S63:begin
        data_28[15:12] = 0;
        data_28[11:8] = data;
        seven_segment_display(data_28[3:0],led);
        seven_segment_display(data_28[7:4],led1);
        seven_segment_display(data_28[11:8],led2);
```

```
        seven_segment_display(data_28[15:12],led3);
        arduino_comm=4'h4;
        next_state = S64;
        tracker = 8'h7F;
end
S64: begin
        data_28[15:12] = data;
        seven_segment_display(data_28[3:0],led);
        seven_segment_display(data_28[7:4],led1);
        seven_segment_display(data_28[11:8],led2);
        seven_segment_display(data_28[15:12],led3);
        arduino_comm=4'h4;
        next_state = S75;
        write_to_memory_2(data_28);
        tracker = 8'h7F;
end
S65: begin
        write_high_2(3'b111);
        mac_rst = 0;
        next_state = S66;
        i=0;
```

```
end
S66: begin
        arduino_comm=4'h8;
        close_write;
        read_high_1(i);
        next_state = S67;
end
S67: begin
        read_from_memory_1(op_data);
        read_high_2(i);
        next_state = S68;
end
S68: begin
        read_from_memory_2(op_data_1);
        i = i+1;
        if(i==8) next_state = S69;
        else next_state = S66;
end
S69: begin
        close_read;
        op_data_1=0;
```

```
        op_data = 0;
        next_state = S70;
end
S70:begin
        tracker = 8'hFF;
        next_state = S71;
        seven_segment_display(result[3:0],led);
        seven_segment_display(result[7:4],led1);
        seven_segment_display(result[11:8],led2);
        seven_segment_display(result[15:12],led3);
 next_state = S71;
 end
S71: next_state = S71;
S72: begin
        if(next_trigger) next_state = S33;
        else next_state = S72;
end
S73: begin
        if(next_trigger) next_state = S65;
        else next_state = S73;
end
```

```
        S74: begin
                if(next_trigger) next_state = S1;
                else next_state = S74;
        end
        S75: begin
                if(next_trigger) next_state = S65;
                else next_state = S75;
        end
        endcase
end
always @(posedge clk2 or posedge rst) begin
        if(rst) current_state <=S0;
        else current_state<=next_state;
end
```

8.1.2 Write to SRAM 1

```
task write_high_1(input [2:0] addr);
        address =addr;
        select = 0;
        write_enable = 0;
endtask
```

8.1.3 Write to SRAM 2

```
task write_high_2(input [2:0] addr);
        address =addr;
        select = 1;
        write_enable = 0;
endtask
```

8.1.4 Close the Writing for SRAM A and B

```
task close_write;
        select = 1;
        write_enable = 1;
endtask
```

8.1.5 Read from SRAM A

```
task read_high_1(input [2:0] addr);
        address =addr;
        select = 0;
        output_enable = 0;
endtask
```

8.1.6 Read from SRAM B

```
task read_high_2(input [2:0] addr);
        address =addr;
        select = 1;
```

```
        output_enable = 0;
endtask
```

8.1.7 Close the Reading for SRAM A and B

```
task close_read;
        select = 1;
        output_enable = 1;
endtask
```

8.1.8 Pushing Data into SRAM A

```
task write_to_memory_1(input [15:0] input_data);
        data_into_sram1= input_data;
endtask
```

8.1.9 Pushing Data into SRAM B

```
task write_to_memory_2(input [15:0] input_data);
        data_into_sram2= input_data;
endtask
```

8.1.10 Extracting Data from SRAM A

```
task read_from_memory_1(output reg[15:0] output_data);
        output_data =data_from_sram1;
endtask
```

8.1.11 Extracting Data from SRAM B

```
task read_from_memory_2(output reg[15:0] output_data);
```

```
        output_data =data_from_sram2;
endtask
```

8.1.12 Displaying BCD Data on 7 Segment Display

```
task seven_segment_display(input [3:0] number ,output reg [7:0] pattern_display );
            case(number)
                        4'h0: pattern_display <= 8'b11000000;
                        4'h1: pattern_display <= 8'b11111001;
                        4'h2: pattern_display <= 8'b10100100;
                        4'h3: pattern_display <= 8'b10110000;
                        4'h4: pattern_display <= 8'b10011001;
                        4'h5: pattern_display <= 8'b10010010;
                        4'h6: pattern_display <= 8'b10000010;
                        4'h7: pattern_display <= 8'b11111000;
                        4'h8: pattern_display <= 8'b10000000;
                        4'h9: pattern_display <= 8'b10011000;
                        4'hA: pattern_display <= 8'b10001000;
                        4'hB: pattern_display <= 8'b10000011;
                        4'hC: pattern_display <= 8'b11000110;
                        4'hD: pattern_display <= 8'b10100001;
                        4'hE: pattern_display <= 8'b10000110;
                        4'hF: pattern_display <= 8'b10001110;
```

```
                    default: pattern_display <= 8'b11000000;
            endcase
endtask
endmodule
```

8.2 Clock Divider for FSM

```
module clock_counter_2(input clk,output reg clk_1);


initial begin
        clk_1 = 0;
end
reg [25:0] counter = 0;


always @(posedge clk) begin


        counter = counter +1;
        if(counter==26'h5) begin
                clk_1 =~clk_1;
                counter=0;
        end
end
endmodule
```

8.3 Clock Divider for Keyboard

```
module clock_counter(input clk ,output reg clk_1);


initial begin
        clk_1 = 0;
end
integer i =0;


always @(posedge clk) begin


        i = i+1;
        if(i==3) begin
                clk_1 =~clk_1;
                i=0;
        end
end
endmodule
```

8.4 Demux

```
module demux(input select_signal,output reg select_output1,select_output2);
        always @* begin
                if(select_signal==1'b1) select_output1 <= 1'b1;
```

```
            else select_output1 <= 1'b0;
        select_output2 <= ~select_output1;
        end
endmodule
```

8.5 Keyboard Scanner

```
module keyboardScanner (input clk, input wire [3:0] col,output reg [3:0] row,
output reg [3:0] keyHex);
reg [1:0] state=2'b00;
reg [1:0] nextState=2'b00;
reg [7:0] keyCode;
//state register
always @(posedge clk)  begin
        state <= nextState;
end
//keyHexput CL
always@(posedge clk) begin
        case (state)
        2'b00: row <= 4'b1110;
        2'b01: row <= 4'b1101;
        2'b10: row <= 4'b1011;
        2'b11: row <= 4'b0111;
        default: row <= 4'b1110;
```

```
endcase
        if (col != 4'b0000) begin
        keyCode <= {row[3],row[2],row[1],row[0],col[3],col[2],col[1],col[0]};
        case(keyCode)
        8'h77: keyHex=4'h1;
        8'h7B: keyHex=4'h4;
        8'h7D: keyHex=4'h7;
        8'h7E: keyHex=4'hF;
        8'hB7: keyHex=4'h2;
        8'hBB: keyHex=4'h5;
        8'hBD: keyHex=4'h8;
        8'hBE: keyHex=4'h0;
        8'hD7: keyHex=4'h3;
        8'hDB: keyHex=4'h6;
        8'hDD: keyHex=4'h9;
        8'hDE: keyHex=4'hE;
        8'hE7: keyHex=4'hA;
        8'hEB: keyHex =4'hB;
        8'hED: keyHex =4'hC;
        8'hEE: keyHex=4'hD;
                endcase
```

```
        end
end

//next state CL
always @(posedge clk) begin
        case (state)
        2'b00: nextState <=2'b01;
        2'b01: nextState <= 2'b10;
        2'b10: nextState <=2'b11;
        2'b11: nextState <=2'b00;
        default: nextState <=2'b00;
        endcase
end

endmodule
```

8.6 Two Stage Pipelined MAC

```
module MAC_unit(input clk,rst, input [15:0] In1,In2, output reg [15:0] result);
reg signed [15:0] mult;
reg input1, input2;
initial begin
        mult <=0;
```

```
        result <=0;
end
always @(In2) begin
        if(rst) begin
                        input1 <=0;
                       input2 <=-0;
                result<=0;
                mult <=0;
        end
        else begin
                        input1<=In1;
                        input2<=In2;
                mult <= multiplication(input1,input2);
                result <= signed_addition(result,mult);
        end
end
```

8.6.2 Signed Addition

```
function [15:0] signed_addition(input [15:0] a,b);
reg overflow;
reg carry;
begin
```

```
        {carry,signed_addition} = a +b;
        overflow = carry ^ signed_addition[15];
        if(overflow) begin
        if({carry,signed_addition[15]}==2'b01) signed_addition= 16'h7FFF;
        else if( {carry,signed_addition[15]}==2'b10) signed_addition = 16'h8001;
        else signed_addition = 16'h0000;
end
end
endfunction
```

8.6.3 Signed Multiplication

```
function [15:0] multiplication(input [15:0] a,b);
reg [31:0] temp;
reg overflow;
begin
        temp = a*b;
        overflow = ^temp[31:24];
        if(overflow) begin
                casex(temp[31:24])
                8'b0xxxxxxx1: multiplication = 16'h7FFF;
                default: multiplication = 16'h8001;
                endcase
```

```
        end
        else begin
                casex(temp[10:0])
                11'bxx0xxxxxxx: multiplication = temp[24:9];
                11'b00100000000: multiplication = temp[24:9];
                default: multiplication = temp[24:9]+16'h1;
                endcase
        end
end
endfunction
endmodule
```

8.7 SRAM Model

```
module sram(input Cs_b, We_b, Oe_b,input [2:0] address,input [15:0] data_in,
output  [15:0]data_out);
reg [15:0] ram_1[8:0];
assign data_out = (Cs_b == 0 && Oe_b ==0) ? ram_1[address]: 16'bz;
always @(*) begin
        if(Cs_b==0 && We_b ==0) begin
                ram_1[address]<=data_in;
        end
end
```

endmodule

8.8 Arduino Code

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>


LiquidCrystal_I2C lcd(0x27,20,4);  // set the LCD address to 0x27 for a 16 chars and 2 line display

int state_0 = 7;

int state_1 = 6;

int state_2 = 5;

int state_3 = 4;

int delay_unit = 5000;

void setup()

{

  Wire.setClock(400000L);

 lcd.init();                      // initialize the lcd

 lcd.backlight();

 pinMode(state_0, INPUT);

 pinMode(state_1,INPUT);

 pinMode(state_2,INPUT);

 pinMode(state_3,INPUT);

 lcd.setCursor(4,0);
```

```
  lcd.print("Group 9");
}
void loop(){
        while(digitalRead(state_0)!=HIGH);
        lcd.clear();
        lcd.setCursor(4,0);
        lcd.print("Group 9");
        lcd.setCursor(4,1);
        lcd.print("Reset");
        while(digitalRead(state_1)!=HIGH);
        lcd.setCursor(2,1);
        lcd.print("SRAM A Input");
        while(digitalRead(state_2)!=HIGH);
        lcd.setCursor(2,1);
        lcd.print("SRAM B Input");
        while(digitalRead(state_3)!=HIGH);
        lcd.clear();
        lcd.setCursor(4,0);
        lcd.print("Group 9");
        lcd.setCursor(2,1);
        lcd.print("FP MAC CALC");
```

```
}
```